

Atividade — Máquinas de Turing

Teoria da Computação — Prof^a Kadidja Valéria

Atividade Data: 29/09/2026

Etapa 1. Introdução

1. O que é uma Máquina de Turing?

É uma construção matemática e abstrata criada por Alan Turing em 1936 para definir, com rigor matemático, o que é a computação e o que significa algo ser "computável"[cite: 2]. Não se trata de uma máquina física, mas de um modelo teórico e minimalista inspirado no funcionamento de uma máquina de escrever [cite: 2]. Ela é composta por uma fita de papel (que serve como memória de tamanho potencialmente infinito), uma cabeça de leitura e escrita que lê e altera um quadrado por vez, e um conjunto finito de estados e regras de transição [cite: 2]. A partir desse modelo cru, Turing provou que qualquer computação que um computador moderno faz pode ser executada por uma Máquina de Turing [cite: 2].

Quais são os principais componentes de uma Máquina de Turing?

- **Fita:** é como se fosse a memória. Ela é dividida em células, cada uma com um símbolo, e é infinita.
- **Cabeça de leitura e escrita:** fica em cima de uma célula por vez. Ela lê o símbolo, pode trocar por outro e anda uma casa para a esquerda ou para a direita.
- **Estados:** mostram em que "momento" do trabalho a máquina está, como se fosse o que ela está lembrando naquele instante.
- **Alfabeto:** são os símbolos que podem aparecer na fita (no meu caso: 0, 1, X, Y e o branco).
- **Regras de transição:** dizem o que a máquina faz de acordo com o estado atual e o símbolo que ela leu.
- **Estado inicial e estados finais:** mostram onde o processamento começa e quando ele termina, aceitando ou rejeitando a entrada.

2. Qual é a importância das Máquinas de Turing para a computação?

Ela é importante porque foi a partir dela que deu para definir, de um jeito formal, o que um computador consegue ou não consegue calcular. Ela é uma das bases da Ciência da Computação e ajudou a formar as ideias de algoritmo e de computabilidade. Pelo vídeo também deu para entender que é isso que separa uma calculadora de um computador: a calculadora só faz as operações que já vêm nela, enquanto um computador, assim como a Máquina de Turing, segue qualquer programa que você der para ele. Essa ideia inspirou a arquitetura de Von Neumann, que é usada até hoje.

3. Qual é a relação entre Máquina de Turing e algoritmo?

A Máquina de Turing é fundamental porque definiu os limites matemáticos da computação (o que é computável e o que não é)[cite: 2]. Além disso, formalizou a ideia da Máquina de Turing Universal: uma máquina projetada para ler o programa e os dados diretamente da fita e simular qualquer outra Máquina de Turing arbitrária[cite: 2]. Conforme explicado no vídeo, essa universalidade é justamente o que separa uma calculadora/máquina de escrever de um computador moderno[cite: 2]: Calculadoras e Máquinas de Calcular Antigas (como Babbage, Eniac, Colossus ou Z3): foram projetadas para tarefas fixas ou exigiam reconfiguração física (cabos, engrenagens ou chaves) para mudar de operação[cite: 2]. Elas não armazenavam o programa junto com os dados[cite: 2]. Computadores Modernos: Baseiam-se na Máquina Universal de Turing e na Arquitetura de Von Neumann, onde dados e programas compartilham o mesmo espaço de memória (a fita/RAM), permitindo carregar e executar qualquer programa de forma dinâmica[cite: 2].

Etapa 2. Simulação

Descrição da Máquina de Turing criada

Montei a máquina no simulador turingmachinesimulator.com para reconhecer palavras do tipo 0ⁿ1ⁿ, ou seja, com a mesma quantidade de 0 e de 1, com todos os 0 vindo antes dos 1.

A ideia é ir "riscando" os símbolos em pares. A máquina marca o primeiro 0 que encontra com um X, anda para a direita até achar o primeiro 1 e marca ele com um Y. Depois ela volta para a esquerda até o último X e repete tudo. Quando não sobra mais nenhum 0, ela passa por cima dos Y para ver se ainda tem algum 1 perdido. Se chegar no final da palavra só com Y, ela aceita. Se em algum momento aparecer um símbolo que não era para estar ali (por exemplo, um 1 sobrando), ela não tem regra para aquilo, então para e rejeita a palavra.

Estados usados: q0 (início de cada rodada), q1 (procurando um 1), q2 (voltando para a esquerda), q3 (conferindo o final) e qA (aceitação).

Tabela de transições

Estado	Lê	Escreve	Move	Vai para	O que significa
q0	0	X	→	q1	Marca um 0 e vai procurar um 1
q0	Y	Y	→	q3	Acabaram os 0: hora de conferir o resto
q1	0	0	→	q1	Passa pelos 0 que ainda faltam
q1	Y	Y	→	q1	Passa pelos 1 já marcados
q1	1	Y	←	q2	Achou um 1, marca e volta
q2	0	0	←	q2	Volta pela fita
q2	Y	Y	←	q2	Volta pela fita
q2	X	X	→	q0	Chegou no último X, começa outra rodada

q3	Y	Y	→	q3	Confere se só sobrou Y
q3	–	–	—	qA	Terminou sem sobras: AC?ITA

Código usado no simulador

name: 0n1n
init: q0
accept: qA

q0,0
q1,X,>

q0,Y
q3,Y,>

q1,0
q1,0,>

q1,Y
q1,Y,>

q1,1
q2,Y,<

q2,0
q2,0,<

q2,Y
q2,Y,<

q2,X
q0,X,>

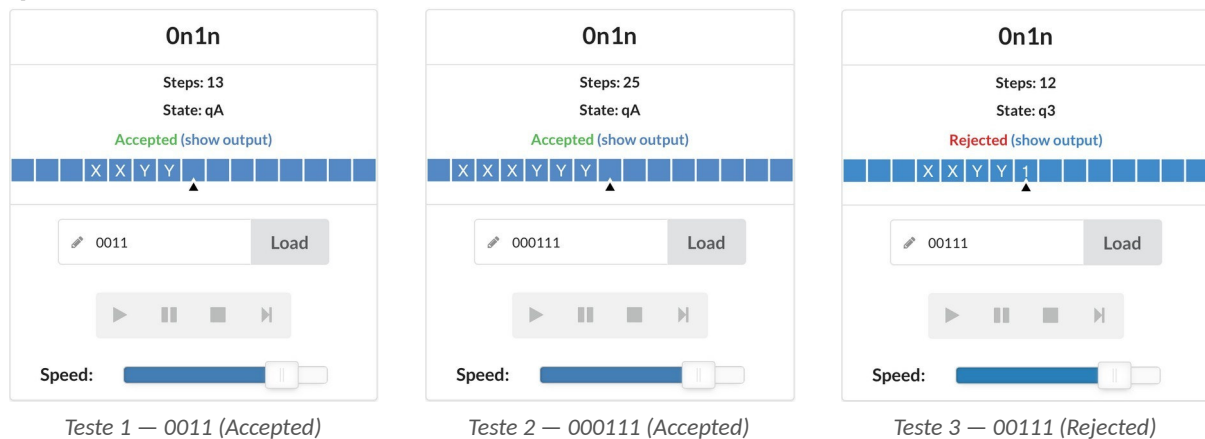
q3,Y
q3,Y,>

q3,_
qA,_, -

Etapa 3. Registro da simulação

Teste	Entrada	Esperado	Obtido	Passos	Estados percorridos
1	0011	AC?ITA	AC?ITA	13	q0 → q1 → q1 → q2 → q2 → q0 → q1 → q1 → q2 → q2 → q0 → q3 → q3 → qA
2	000111	AC?ITA	AC?ITA	25	(q0 → q1 → q1 → q1 → q2 → q2 → q2) repetido 3 vezes → q0 → q3 → q3 → q3 → qA
3	00111	R?J?ITA	R?J?ITA	12	q0 → q1 → q1 → q2 → q2 → q0 → q1 → q1 → q2 → q2 → q0 → q3 → q3 (parou lendo 1)

Capturas de tela



Interpretação dos resultados

Os três testes deram o resultado que eu esperava. No teste 1 (0011) e no teste 2 (000111), a máquina conseguiu formar um par para cada 0 e cada 1, então no final a fita ficou só com X e Y e ela terminou no estado qA, aceitando a palavra. Dá para ver também que quanto maior a palavra, mais passos ela leva (13 para 0011 e 25 para 000111), porque a cabeça precisa ir e voltar pela fita a cada par.

Já no teste 3 (00111) tem um 1 a mais do que 0. Depois de formar os dois pares, sobrou um 1 sem par. A máquina estava no estado q3, que só espera encontrar Y ou o fim da palavra, e como leu um 1 ela parou sem chegar em qA. Por isso a palavra foi rejeitada, que era exatamente o esperado.

Etapa 4. Reflexão sobre os limites computacionais

Não, uma Máquina de Turing não consegue resolver qualquer problema. Parece estranho, porque ela consegue fazer tudo o que um computador faz, mas justamente por isso ela também tem os mesmos limites. O exemplo mais conhecido é o Problema da Parada: não existe nenhum algoritmo que consiga olhar para qualquer programa e dizer com certeza se ele vai terminar ou se vai rodar para sempre. O Turing provou isso mostrando que, se esse "verificador" existisse, daria para criar um programa que usa o verificador contra ele mesmo, e isso gera uma contradição. Ou seja, não é falta de tecnologia ou de computador mais rápido, é uma impossibilidade lógica. Como qualquer computador é, no fundo, equivalente a uma Máquina de Turing, esse limite vale para todos eles.

Questão final

Para saber se um problema é apenas difícil ou se é impossível de resolver, você precisa olhar para a existência de um algoritmo garantido. Se o problema for apenas **difícil**, existe um algoritmo que sempre chega ao resultado correto e termina em um tempo finito, mesmo que o custo de tempo ou memória seja extremamente alto (como ocorre em problemas NP-completos). Por outro lado,

se o problema for **impossível (indecidível)**, a Teoria da Computação demonstra que não existe e nunca existirá nenhum programa capaz de resolvê-lo para todas as entradas possíveis.

A ferramenta teórica usada para definir esse limite é a **Máquina de Turing (MT)**. Pela Tese de Church-Turing, se uma Máquina de Turing não consegue resolver um problema de forma genérica, nenhum computador real jamais conseguirá. Para determinar formalmente que um problema é indecidível, utiliza-se a técnica de **redução**: você prova matematicamente que, se o seu problema tivesse solução algorítmica, ele poderia ser usado para resolver o famoso Problema da Parada. Como já foi demonstrado por Alan Turing que o Problema da Parada é indecidível, o seu problema também se torna comprovadamente impossível.